Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Дисциплина
«Проектирование вычислительных систем»

Лабораторная работа № 1

Интерфейсы ввода-вывода общего назначения (GPIO)

Вариант №1

Выполнили: Малышев Никита Александрович,
Медведев Ярослав Александрович
Группа: P3412
Проверил: Пинкевич Василий Юрьевич

Санкт-Петербург
2026 год

Содержание

1. Задание 3
2. Выполнение 4
   2.1. Используемые контакты 4
   2.2. Логика работы программы 4
   2.3. Архитектура программы 6
   2.4. Блок-схемы алгоритма 8
   2.5. Драйвер GPIO 13
3. Исходные коды 14
   3.1. gpio_driver.h 14
   3.2. gpio_driver.c 15
   3.3. board_config.h 17
   3.4. led.h 18
   3.5. led.c 18
   3.6. button.h 19
   3.7. button.c 19
   3.8. traffic_light.h 21
   3.9. traffic_light.c 21
   3.10. main.c (пользовательские части) 23
4. Выводы 23

1. Задание

Сымитировать работу светофора для автомобилей, которые останавливаются на пешеходном переходе для пропуска людей. Светофор циклически переключает цвета в следующем порядке (порядок условный, соответствие реальному светофору не соблюдается): красный, зелёный, зелёный мигающий, жёлтый, снова красный и т. д. По умолчанию период горения красного составляет 12 секунд. Зеленый должен гореть в четыре раза меньше времени. Если во время горения зеленого мигающего, желтого или красного нажимается кнопка, светофор запоминает необходимость скорейшего переключения на зелёный. После нажатия кнопки последовательность переключения цветов не нарушается, но ближайший период горения красного должен быть сокращен до ¼ своего обычного периода. Если кнопка нажата во время горения красного, и при этом до конца горения осталось более ¼ периода, оставшееся время горения должно быть сокращено до ¼ периода. Если времени осталось меньше ¼, ничего менять не требуется. Следующий красный должен снова гореть полную длительность, если только снова не будет нажата кнопка.

Разработать собственную реализации драйвера GPIO (не HAL и LL) и внедрить ее в разработанную ранее программу для стенда.

2. Выполнение

2.1. Используемые контакты

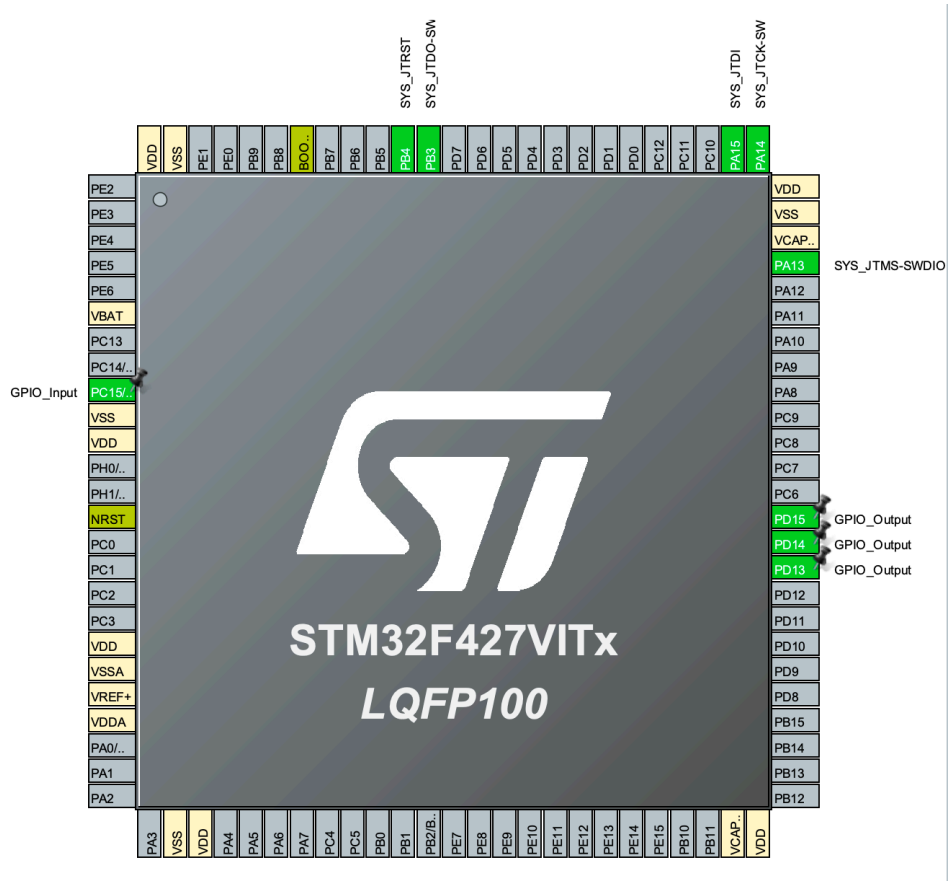


Рис. 1. Используемые контакты

- PC15 - перехватывает нажатие кнопки (настроен на GPIO_Input)
- PD13 - управляет зеленым светодиодом (настроен на GPIO_Output)
- PD14 - управляет желтым светодиодом (настроен на GPIO_Output)
- PD15 - управляет красным светодиодом (настроен на GPIO_Output)
- PB3, PB4, PA13, PA14, PA15 - J-Tag для отладки

2.2. Логика работы программы

Светофор работает как конечный автомат из четырёх состояний: красный, зелёный, зелёный мигающий и жёлтый. Переходы между состояниями выполняются по истечении времени горения текущего цвета в порядке «красный, зелёный, зелёный мигающий, жёлтый, красный и т. д.». После включения питания автомат начинает работу с красного.

Длительности состояний приведены в таблице. В задании длительности зелёного мигающего и жёлтого не указаны, они приняты равными 2 с. Мигающий зелёный горит 250 мс и гаснет на 250 мс.

Табл. 1. Длительности состояний светофора

Состояние	**Светодиод**	**Длительность**
Красный	красный	12 с, при сокращении 3 с (¼ периода)
Зелёный	зелёный	3 с (в четыре раза меньше красного)
Зелёный мигающий	зелёный с периодом 500 мс	2 с
Жёлтый	жёлтый	2 с

Реакция на нажатие кнопки зависит от состояния, в котором светофор находился в момент нажатия.

Табл. 2. Реакция светофора на нажатие кнопки

Состояние в момент нажатия	**Реакция**
Красный, до конца осталось более 3 с	оставшееся время горения сокращается до 3 с
Красный, до конца осталось не более 3 с	ничего не меняется
Зелёный мигающий или жёлтый	запоминается заявка, ближайший красный горит 3 с
Зелёный	нажатие не запоминается

Запомненная заявка сбрасывается при входе в красный, поэтому следующий красный снова горит полную длительность, если кнопку не нажимали повторно. При нажатии на красный заявка не запоминается: сокращается только текущий период.

Программная защита от дребезга построена на измерении времени: изменение уровня на входе считается нажатием, только если новый уровень остаётся неизменным не менее 30 мс. Нажатие фиксируется как одно событие на каждое срабатывание кнопки.

Программа не использует блокирующих ожиданий, прерываний и дополнительных таймеров, кроме SysTick. Параллельные процессы (опрос кнопки, мигание, отсчёт времени горения) выполняются по принципу кооперативной многозадачности: главный цикл поочерёдно вызывает функции `button_update()` и `traffic_light_update()`, каждая из которых сравнивает текущее значение `HAL_GetTick()` с моментом начала ожидания и сразу возвращает управление.

2.3. Архитектура программы

Программа разделена на модули по принципу минимальной взаимозависимости. Прикладная логика не содержит обращений к аппаратуре: номера портов и контактов скрыты в `board_config.h`, а работа с регистрами выполняется только в драйвере GPIO.



Листинг 1. Уровни программы

Модули `led.c` и `button.c` получают привязку к контактам из `board_config.h`. Модули `button.c` и `traffic_light.c` измеряют время через `HAL_GetTick()`, то есть через счётчик SysTick. Все остальные обращения к GPIO выполняются только через `gpio_driver`.

Табл. 3. Модули программы и их связи

Модуль	Назначение	Зависит от
`board_config.h`	привязка светодиодов и кнопки к контактам, уровни включения и нажатия	`gpio_driver.h`
`gpio_driver.c/.h`	собственный драйвер GPIO	`GPIO_TypeDef`, регистры RCC
`led.c/.h`	драйвер светодиодов: включение, выключение, переключение	`gpio_driver`, `board_config`
`button.c/.h`	драйвер кнопки с защитой от дребезга и событием нажатия	`gpio_driver`, `board_config`, `HAL_GetTick`
`traffic_light.c/.h`	прикладная логика светофора, неблокирующий автомат	`led`, `button`, `HAL_GetTick`
`main.c`	инициализация модулей и главный цикл	`led`, `button`, `traffic_light`

Драйверы светодиодов и кнопки универсальны и не содержат прикладной логики: `led_set()`, `led_toggle()` и `led_all_off()` работают с любым из светодиодов по идентификатору, `button_pressed_event()` возвращает истину один раз на каждое подтверждённое нажатие. Функции драйверов не блокируют выполнение и не содержат пауз с активным ожиданием.

Главный цикл программы состоит из двух вызовов:

```
while (1)
{
    button_update();
    traffic_light_update();
}
```

2.4. Блок-схемы алгоритма

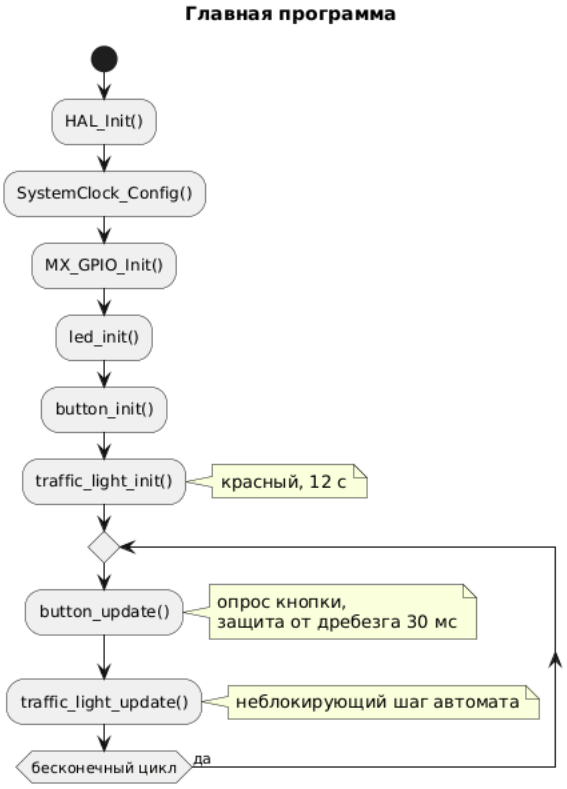


Рис. 2. Главная программа

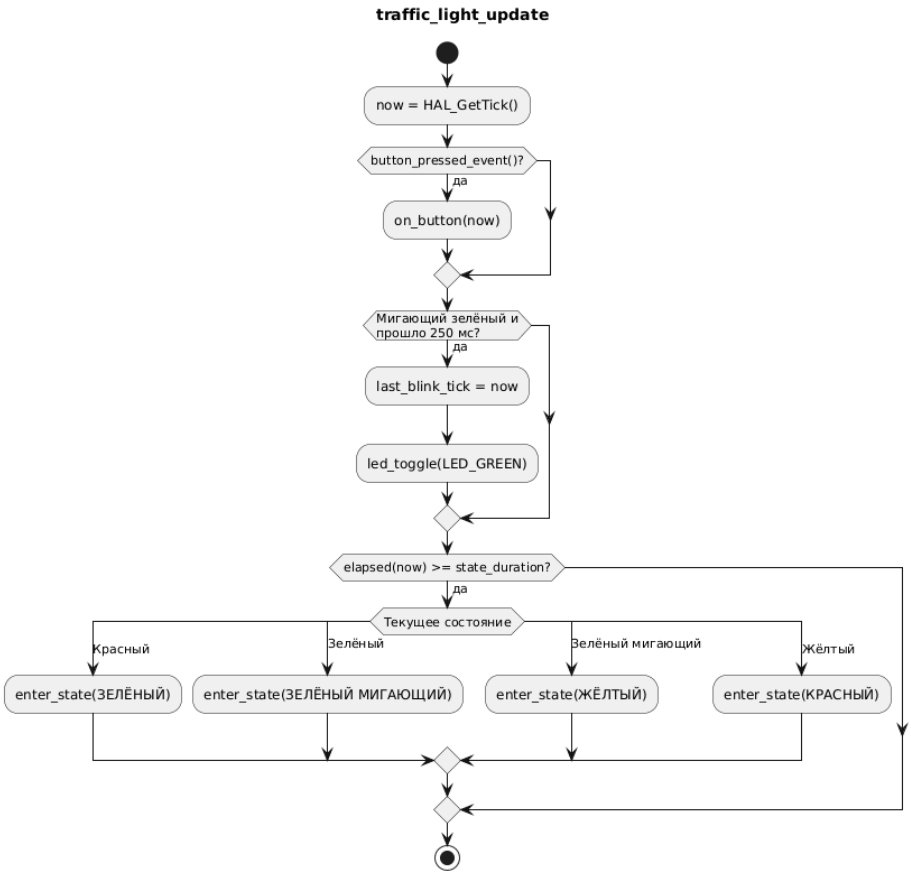


Рис. 3. Шаг светофора, функция `traffic_light_update()`

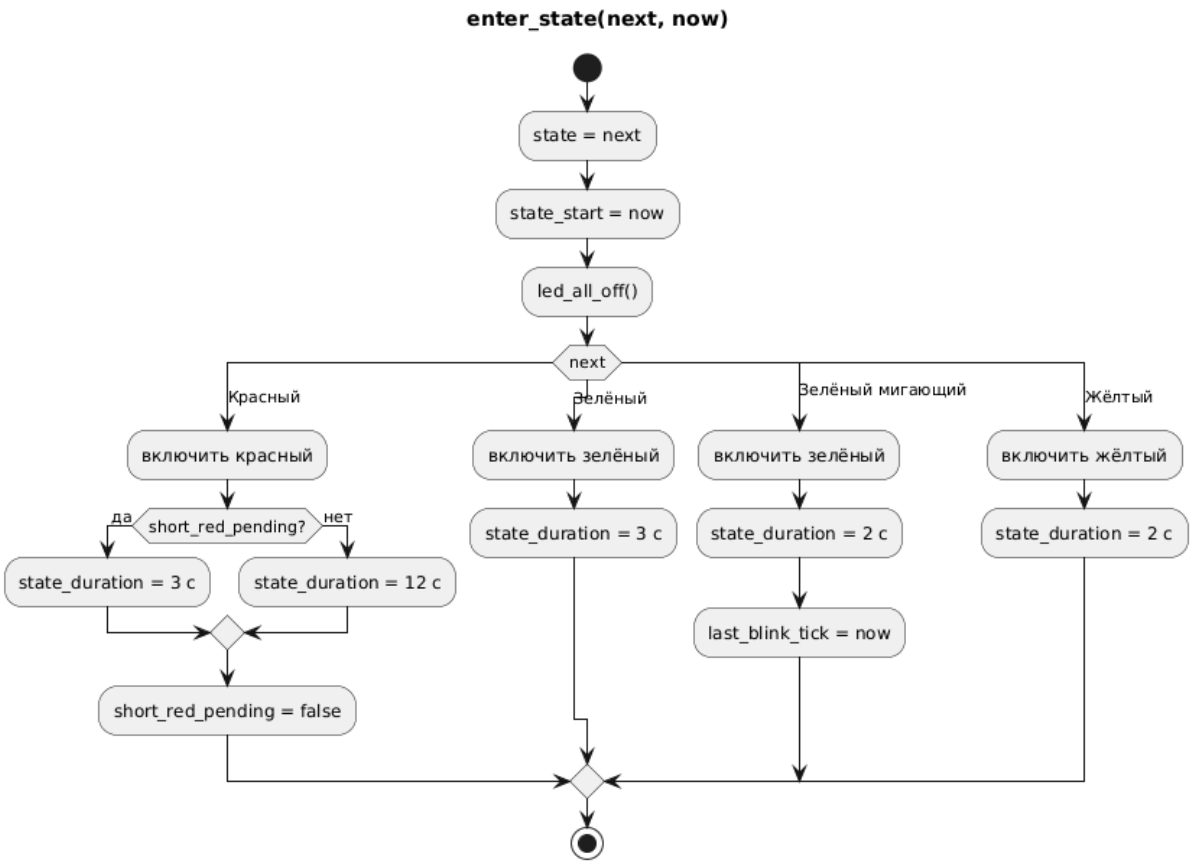


Рис. 4. Переход в состояние, функция `enter_state()`

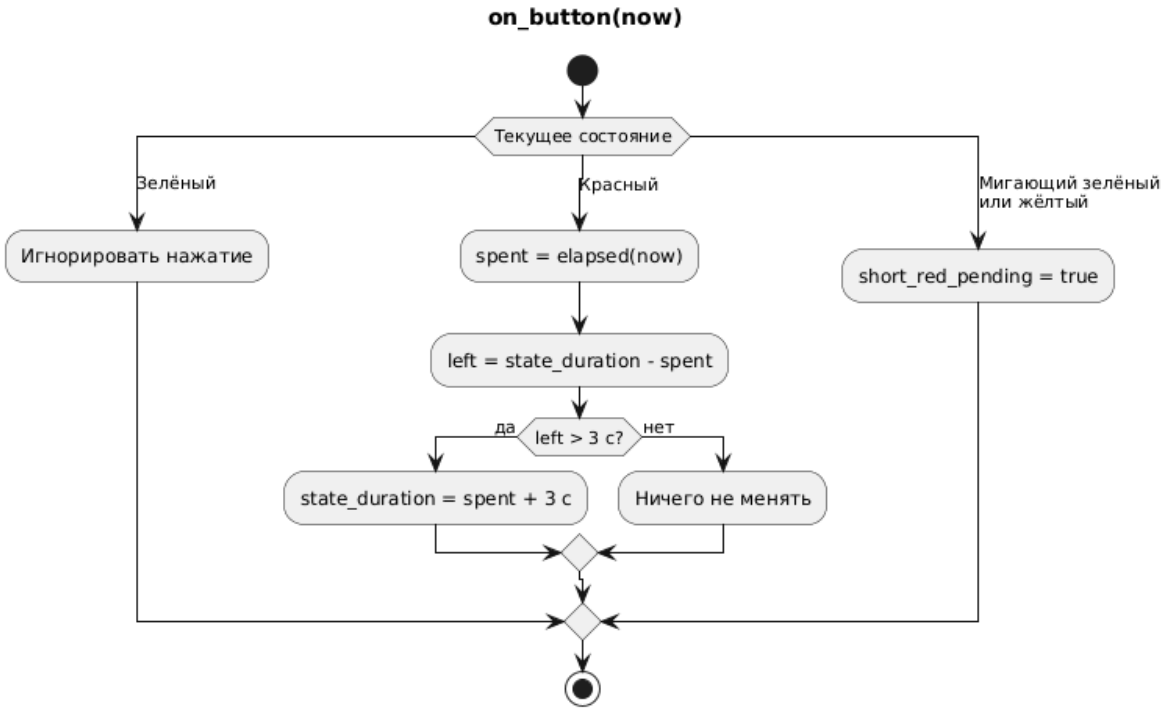


Рис. 5. Обработка нажатия кнопки, функция `on_button()`

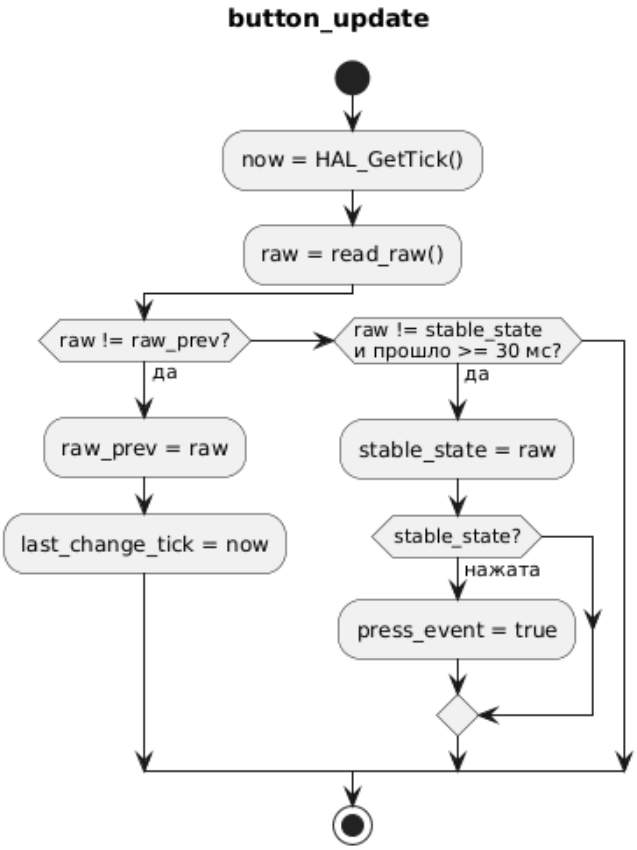


Рис. 6. Опрос кнопки с защитой от дребезга, функция `button_update()`

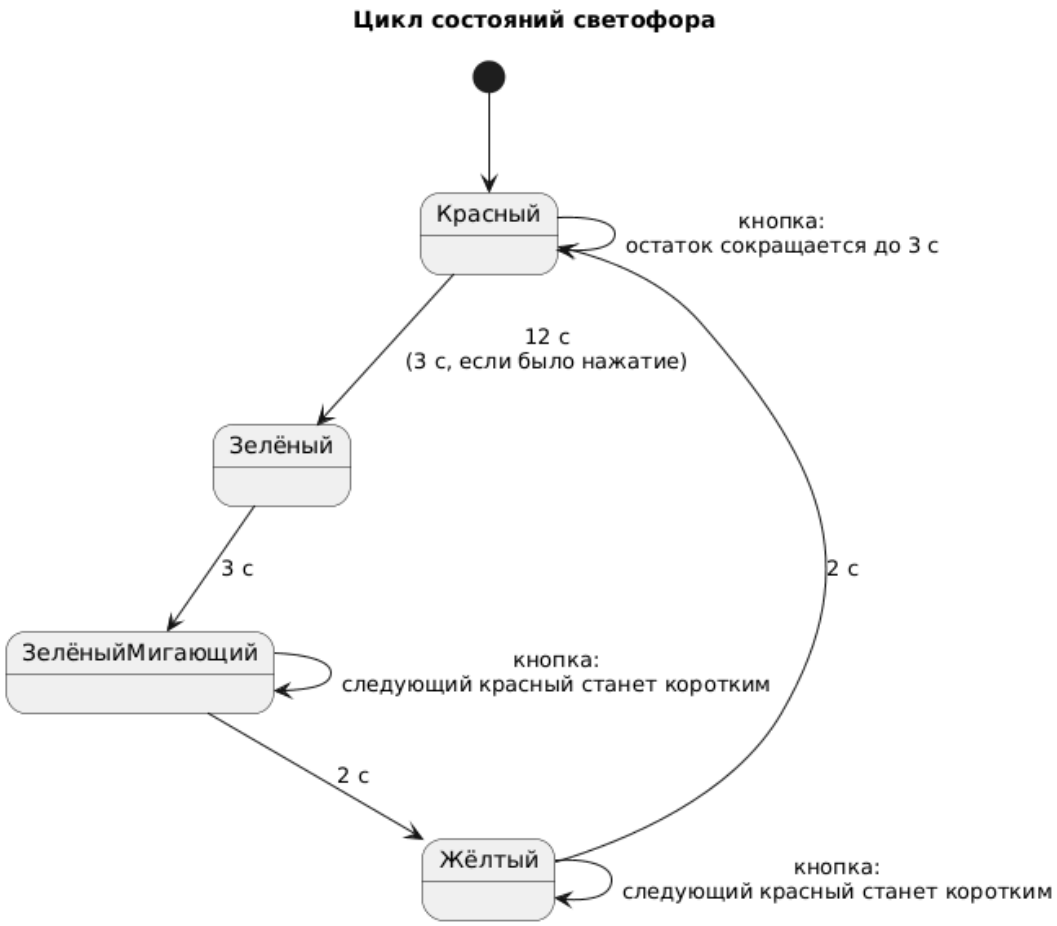


Рис. 7. Диаграмма состояний светофора

2.5. Драйвер GPIO

Табл. 4. Функции драйвера GPIO

Функция	**Назначение**
`gpio_init(port, pins, config)`	инициализация режима работы одного или нескольких выводов порта, включение тактирования порта
`gpio_get_config(port, pin, config)`	получение текущих настроек вывода: режим, тип выхода, скорость, подтяжка, альтернативная функция, уровень выходного регистра
`gpio_read_port(port)`	чтение всех 16 входных сигналов порта
`gpio_read_pins(port, mask)`	чтение нескольких сигналов порта по маске
`gpio_read_pin(port, pin)`	чтение одного сигнала
`gpio_write_pins(port, mask, value)`	одновременная запись произвольных значений в несколько выходов порта, остальные выходы не затрагиваются
`gpio_write_pin(port, pin, level)`	запись одного сигнала
`gpio_toggle_pins(port, mask)`	инвертирование нескольких выходов

Параметры вывода описываются структурой `gpio_config_t`:

Табл. 5. Поля структуры `gpio_config_t`

Поле	**Значения**
`mode`	вход, выход, альтернативная функция, аналоговый режим
`otype`	push-pull или открытый сток
`speed`	низкая, средняя, высокая, очень высокая
`pull`	без подтяжки, к питанию, к земле
`alt_func`	номер альтернативной функции 0–15
`level`	начальный выходной уровень: низкий, высокий или оставить текущий

3. Исходные коды

3.1. gpio_driver.h

```
#ifndef GPIO_DRIVER_H
#define GPIO_DRIVER_H

#include <stdbool.h>
#include <stdint.h>
#include "main.h"

typedef enum {
    PIN_MODE_INPUT  = 0U,
    PIN_MODE_OUTPUT = 1U,
    PIN_MODE_ALT    = 2U,
    PIN_MODE_ANALOG = 3U
} gpio_mode_t;

typedef enum {
    PIN_OTYPE_PUSH_PULL  = 0U,
    PIN_OTYPE_OPEN_DRAIN = 1U
} gpio_otype_t;

typedef enum {
    PIN_SPEED_LOW       = 0U,
    PIN_SPEED_MEDIUM    = 1U,
    PIN_SPEED_HIGH      = 2U,
    PIN_SPEED_VERY_HIGH = 3U
} gpio_speed_t;

typedef enum {
    PIN_PULL_NONE = 0U,
    PIN_PULL_UP   = 1U,
    PIN_PULL_DOWN = 2U
} gpio_pull_t;

typedef enum {
    PIN_LEVEL_LOW  = 0U,
    PIN_LEVEL_HIGH = 1U,
    PIN_LEVEL_KEEP = 2U
} gpio_level_t;

typedef struct {
    gpio_mode_t  mode;
    gpio_otype_t otype;
    gpio_speed_t speed;
    gpio_pull_t  pull;
    uint8_t      alt_func;
    gpio_level_t level;
} gpio_config_t;

void     gpio_init(GPIO_TypeDef *port, uint16_t pins, const gpio_config_t
*config);
bool     gpio_get_config(GPIO_TypeDef *port, uint16_t pin, gpio_config_t
```

```
*config);

uint16_t gpio_read_port(GPIO_TypeDef *port);
uint16_t gpio_read_pins(GPIO_TypeDef *port, uint16_t mask);
bool     gpio_read_pin(GPIO_TypeDef *port, uint16_t pin);

void     gpio_write_pins(GPIO_TypeDef *port, uint16_t mask, uint16_t value);
void     gpio_write_pin(GPIO_TypeDef *port, uint16_t pin, bool level);
void     gpio_toggle_pins(GPIO_TypeDef *port, uint16_t mask);

#endif
```

3.2. gpio_driver.c

```
#include "gpio_driver.h"
#include <stddef.h>

static bool is_single_pin(uint16_t pin)
{
    return (pin != 0U) && ((pin & (uint16_t)(pin - 1U)) == 0U);
}

static uint32_t pin_number(uint16_t pin)
{
    uint32_t n = 0U;
    while (((uint32_t)pin >> n) != 1U) {
        n++;
    }
    return n;
}

static void clock_enable(GPIO_TypeDef *port)
{
    uint32_t bit = 1UL << (((uint32_t)port - (uint32_t)GPIOA_BASE) >> 10U);
#if defined(RCC_AHBENR_GPIOAEN)
    RCC->AHBENR |= bit;
    (void)RCC->AHBENR;
#else
    RCC->AHB1ENR |= bit;
    (void)RCC->AHB1ENR;
#endif
}

static void set_fields(volatile uint32_t *reg, uint32_t pins, uint32_t width,
uint32_t value)
{
    uint32_t field = (1UL << width) - 1UL;
    uint32_t mask  = 0U;
    uint32_t bits  = 0U;

    for (uint32_t i = 0U; i < (32U / width); i++) {
        if ((pins & (1UL << i)) != 0U) {
            mask |= field << (i * width);
```

```
            bits |= (value & field) << (i * width);
        }
    }
    *reg = (*reg & ~mask) | bits;
}

static void write_masked(GPIO_TypeDef *port, uint16_t mask, uint16_t value)
{
    uint32_t set   = (uint32_t)(value & mask);
    uint32_t reset = (uint32_t)((uint16_t)(~value) & mask);
    port->BSRR = (reset << 16U) | set;
}

void gpio_init(GPIO_TypeDef *port, uint16_t pins, const gpio_config_t *config)
{
    if ((port == NULL) || (pins == 0U) || (config == NULL)) {
        return;
    }

    clock_enable(port);

    gpio_pull_t pull = (config->mode == PIN_MODE_ANALOG) ? PIN_PULL_NONE :
config->pull;

    if ((config->mode == PIN_MODE_OUTPUT) && (config->level != PIN_LEVEL_KEEP))
{
        write_masked(port, pins, (config->level == PIN_LEVEL_HIGH) ? pins :
(uint16_t)0U);
    }

    if ((config->mode == PIN_MODE_OUTPUT) || (config->mode == PIN_MODE_ALT)) {
        set_fields(&port->OTYPER,  pins, 1U, (uint32_t)config->otype);
        set_fields(&port->OSPEEDR, pins, 2U, (uint32_t)config->speed);
    }

    set_fields(&port->PUPDR, pins, 2U, (uint32_t)pull);

    if (config->mode == PIN_MODE_ALT) {
        set_fields(&port->AFR[0], (uint32_t)pins & 0x00FFU, 4U, config-
>alt_func);
        set_fields(&port->AFR[1], (uint32_t)pins >> 8U,     4U, config-
>alt_func);
    }

    set_fields(&port->MODER, pins, 2U, (uint32_t)config->mode);
}

bool gpio_get_config(GPIO_TypeDef *port, uint16_t pin, gpio_config_t *config)
{
    if ((port == NULL) || (config == NULL) || !is_single_pin(pin)) {
        return false;
    }

    uint32_t n = pin_number(pin);
```

```
    config->mode     = (gpio_mode_t)((port->MODER >> (2U * n)) & 0x3U);
    config->otype    = (gpio_otype_t)((port->OTYPER >> n) & 0x1U);
    config->speed    = (gpio_speed_t)((port->OSPEEDR >> (2U * n)) & 0x3U);
    config->pull     = (gpio_pull_t)((port->PUPDR >> (2U * n)) & 0x3U);
    config->alt_func = (uint8_t)((port->AFR[n >> 3U] >> ((n & 7U) * 4U)) &
0xFU);
    config->level    = (((port->ODR >> n) & 0x1U) != 0U) ? PIN_LEVEL_HIGH :
PIN_LEVEL_LOW;
    return true;
}

uint16_t gpio_read_port(GPIO_TypeDef *port)
{
    return (uint16_t)port->IDR;
}

uint16_t gpio_read_pins(GPIO_TypeDef *port, uint16_t mask)
{
    return (uint16_t)(port->IDR & mask);
}

bool gpio_read_pin(GPIO_TypeDef *port, uint16_t pin)
{
    return (port->IDR & pin) != 0U;
}

void gpio_write_pins(GPIO_TypeDef *port, uint16_t mask, uint16_t value)
{
    write_masked(port, mask, value);
}

void gpio_write_pin(GPIO_TypeDef *port, uint16_t pin, bool level)
{
    write_masked(port, pin, level ? pin : (uint16_t)0U);
}

void gpio_toggle_pins(GPIO_TypeDef *port, uint16_t mask)
{
    uint32_t odr = port->ODR;
    uint32_t m   = (uint32_t)mask;
    port->BSRR = ((odr & m) << 16U) | (~odr & m);
}
```

3.3. board_config.h

```
#ifndef BOARD_CONFIG_H
#define BOARD_CONFIG_H

#include <stdbool.h>
#include "main.h"
#include "gpio_driver.h"
```

```
#define LED_GREEN_PORT        GPIOD
#define LED_GREEN_PIN         GPIO_PIN_13
#define LED_YELLOW_PORT       GPIOD
#define LED_YELLOW_PIN        GPIO_PIN_14
#define LED_RED_PORT          GPIOD
#define LED_RED_PIN           GPIO_PIN_15

#define LED_ON_LEVEL          true

#define BUTTON_PORT           GPIOC
#define BUTTON_PIN            GPIO_PIN_15
#define BUTTON_ACTIVE_LEVEL   false
#define BUTTON_PULL           PIN_PULL_UP

#endif
```

3.4. led.h

```
#ifndef LED_H
#define LED_H

#include <stdbool.h>

typedef enum {
    LED_RED,
    LED_YELLOW,
    LED_GREEN,
    LED_COUNT
} led_t;

void led_init(void);
void led_set(led_t led, bool on);
void led_toggle(led_t led);
void led_all_off(void);

#endif
```

3.5. led.c

```
#include "led.h"
#include "board_config.h"

typedef struct {
    GPIO_TypeDef *port;
    uint16_t      pin;
} led_desc_t;

static const led_desc_t leds[LED_COUNT] = {
    [LED_RED]    = { LED_RED_PORT,    LED_RED_PIN    },
    [LED_YELLOW] = { LED_YELLOW_PORT, LED_YELLOW_PIN },
    [LED_GREEN]  = { LED_GREEN_PORT,  LED_GREEN_PIN  },
};
```

```
void led_init(void)
{
    gpio_config_t config = {
        .mode     = PIN_MODE_OUTPUT,
        .otype    = PIN_OTYPE_PUSH_PULL,
        .speed    = PIN_SPEED_LOW,
        .pull     = PIN_PULL_NONE,
        .alt_func = 0U,
        .level    = LED_ON_LEVEL ? PIN_LEVEL_LOW : PIN_LEVEL_HIGH
    };

    for (int i = 0; i < LED_COUNT; i++) {
        gpio_init(leds[i].port, leds[i].pin, &config);
    }
}

void led_set(led_t led, bool on)
{
    bool level = on ? LED_ON_LEVEL : !LED_ON_LEVEL;
    gpio_write_pin(leds[led].port, leds[led].pin, level);
}

void led_toggle(led_t led)
{
    gpio_toggle_pins(leds[led].port, leds[led].pin);
}

void led_all_off(void)
{
    for (int i = 0; i < LED_COUNT; i++) {
        led_set((led_t)i, false);
    }
}
```

3.6. button.h

```
#ifndef BUTTON_H
#define BUTTON_H

#include <stdbool.h>

void button_init(void);
void button_update(void);
bool button_pressed_event(void);

#endif
```

3.7. button.c

```
#include "button.h"
#include "board_config.h"
```

```c
#define BUTTON_DEBOUNCE_MS  30U

static bool     raw_prev;
static bool     stable_state;
static bool     press_event;
static uint32_t last_change_tick;

static bool read_raw(void)
{
    return gpio_read_pin(BUTTON_PORT, BUTTON_PIN) == BUTTON_ACTIVE_LEVEL;
}

void button_init(void)
{
    gpio_config_t config = {
        .mode     = PIN_MODE_INPUT,
        .otype    = PIN_OTYPE_PUSH_PULL,
        .speed    = PIN_SPEED_LOW,
        .pull     = BUTTON_PULL,
        .alt_func = 0U,
        .level    = PIN_LEVEL_KEEP
    };
    gpio_init(BUTTON_PORT, BUTTON_PIN, &config);

    raw_prev         = read_raw();
    stable_state     = raw_prev;
    press_event      = false;
    last_change_tick = HAL_GetTick();
}

void button_update(void)
{
    uint32_t now = HAL_GetTick();
    bool raw = read_raw();

    if (raw != raw_prev) {
        raw_prev = raw;
        last_change_tick = now;
    } else if (raw != stable_state &&
               (uint32_t)(now - last_change_tick) >= BUTTON_DEBOUNCE_MS) {
        stable_state = raw;
        if (stable_state) {
            press_event = true;
        }
    }
}

bool button_pressed_event(void)
{
    bool event = press_event;
    press_event = false;
    return event;
}
```

3.8. traffic_light.h

```h
#ifndef TRAFFIC_LIGHT_H
#define TRAFFIC_LIGHT_H

void traffic_light_init(void);
void traffic_light_update(void);

#endif
```

3.9. traffic_light.c

```c
#include "traffic_light.h"
#include "led.h"
#include "button.h"
#include "main.h"

#define RED_MS            12000U
#define RED_SHORT_MS      (RED_MS / 4U)
#define GREEN_MS          (RED_MS / 4U)
#define GREEN_BLINK_MS    2000U
#define BLINK_HALF_MS     250U
#define YELLOW_MS         2000U

typedef enum {
    TL_RED,
    TL_GREEN,
    TL_GREEN_BLINK,
    TL_YELLOW
} tl_state_t;

static tl_state_t state;
static uint32_t   state_start;
static uint32_t   state_duration;
static uint32_t   last_blink_tick;
static bool       short_red_pending;

static uint32_t elapsed(uint32_t now)
{
    return (uint32_t)(now - state_start);
}

static void enter_state(tl_state_t next, uint32_t now)
{
    state       = next;
    state_start = now;
    led_all_off();

    switch (next) {
    case TL_RED:
        led_set(LED_RED, true);
        state_duration    = short_red_pending ? RED_SHORT_MS : RED_MS;
        short_red_pending = false;
```

```
        break;
    case TL_GREEN:
        led_set(LED_GREEN, true);
        state_duration = GREEN_MS;
        break;
    case TL_GREEN_BLINK:
        led_set(LED_GREEN, true);
        state_duration  = GREEN_BLINK_MS;
        last_blink_tick = now;
        break;
    case TL_YELLOW:
        led_set(LED_YELLOW, true);
        state_duration = YELLOW_MS;
        break;
    }
}

static void on_button(uint32_t now)
{
    switch (state) {
    case TL_GREEN:
        break;

    case TL_RED: {
        uint32_t spent = elapsed(now);
        if (spent < state_duration) {
            uint32_t left = state_duration - spent;
            if (left > RED_SHORT_MS) {
                state_duration = spent + RED_SHORT_MS;
            }
        }
        break;
    }

    case TL_GREEN_BLINK:
    case TL_YELLOW:
        short_red_pending = true;
        break;
    }
}

void traffic_light_init(void)
{
    short_red_pending = false;
    enter_state(TL_RED, HAL_GetTick());
}

void traffic_light_update(void)
{
    uint32_t now = HAL_GetTick();

    if (button_pressed_event()) {
        on_button(now);
    }
```

```
    if (state == TL_GREEN_BLINK &&
        (uint32_t)(now - last_blink_tick) >= BLINK_HALF_MS) {
        last_blink_tick = now;
        led_toggle(LED_GREEN);
    }

    if (elapsed(now) >= state_duration) {
        switch (state) {
        case TL_RED:         enter_state(TL_GREEN, now);       break;
        case TL_GREEN:       enter_state(TL_GREEN_BLINK, now); break;
        case TL_GREEN_BLINK: enter_state(TL_YELLOW, now);      break;
        case TL_YELLOW:      enter_state(TL_RED, now);         break;
        }
    }
}
```

3.10. main.c (пользовательские части)

```
/* USER CODE BEGIN Includes */
#include "led.h"
#include "button.h"
#include "traffic_light.h"
/* USER CODE END Includes */

  /* USER CODE BEGIN 2 */
  led_init();
  button_init();
  traffic_light_init();
  /* USER CODE END 2 */

  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    button_update();
    traffic_light_update();
  }
  /* USER CODE END 3 */
```

4. Выводы

В ходе лабораторной работы изучено устройство интерфейса GPIO микроконтроллера. Разработан собственный драйвер GPIO, не использующий функции HAL и LL. Реализована программа светофора в виде неблокирующего конечного автомата.